

MODULE 1: DATABASE DESIGN AND IMPLEMENTATION

UNIT 3: DATABASE SYSTEM CATALOG

3.0 INTRODUCTION:

A relational database system needs to maintain data about the relations, such as the schema of the relations. In general, such “data about data” is referred to as metadata. Relational schemas and other metadata about relations are stored in a structure called the data dictionary or system catalog. The system catalogue is a collection of tables and views that contain important information about a database. It is the place where a relational database management system stores schema metadata, such as information about tables and columns, and internal bookkeeping information.

4.1 Objectives:

After going through this unit, you should be able to:

1. define the system catalogue and its content;
2. describe the use of catalogue in a commercial DBMS;
3. define the data dictionary system and its advantages and disadvantages, and
4. define the role of catalogue in system administration

3.2 What is a database system catalog?

As explained earlier, one of the responsibilities of DBMS is to provide a system catalog or data dictionary function regarding the various objects that are of interest to the system itself. The database system catalog is a collection of tables and views that contain important information about a database. It is the place where a relational database management system stores schema metadata, such as information about tables and columns, and internal bookkeeping information. A system catalogue is available for each database. Information in the system catalog defines the structure of the database. For example, the DDL for all tables in the database is stored in the system catalog. Most system catalogues are copied from the template database during database creation, and

are thereafter database-specific. A few catalogues are physically shared across all databases in an installation; these are marked in the descriptions of the individual catalogues.

Among the types of information that the system must store are the following:

1. Names of the relations.
2. Names of the attributes of each relation.
3. Domains and lengths of attributes.
4. Names of views defined on the database, and definitions of those views.
5. Integrity constraints (for example, key constraints).

In addition, many systems keep the following data on users of the system:

1. Names of authorized users.
2. Authorization and accounting information about users.
3. Passwords or other information used to authenticate users.

Moreover, the database may store statistical and descriptive data about the relations, such as:

- Number of tuples in each relation.
- Method of storage for each relation (for example, clustered or non-clustered).

The data dictionary may also note the storage organization (sequential, hash, or heap) of relations, and the location where each relation is stored:

1. If relations are stored in operating system files, the dictionary would note the names of the file (or files) containing each relation.
2. If the database stores all relations in a single file, the dictionary may note the blocks containing records of each relation in a data structure such as a linked list.
3. Name of the index.
4. Name of the relation being indexed.
5. Attributes on which the index is defined.
6. Type of index formed

The exact choice of how to represent system metadata by relations must be made by the system designers. One possible representation, with primary keys underlined, is shown in In this representation, the attribute index attributes of the relation Index metadata is assumed to contain a list of one or more attributes, which can be represented by a character string such as “dept name, building”. The Index metadata relation is thus not in first normal form; it can be normalized, but the above representation is likely to be more efficient to access. The data dictionary is often stored in a non-normalized form to achieve fast access.

Whenever the database system needs to retrieve records from a relation, it must first consult the Relation metadata relation to find the location and storage organization of the relation, and then fetch records using this information. However, the storage organization and location of the Relation metadata relation itself

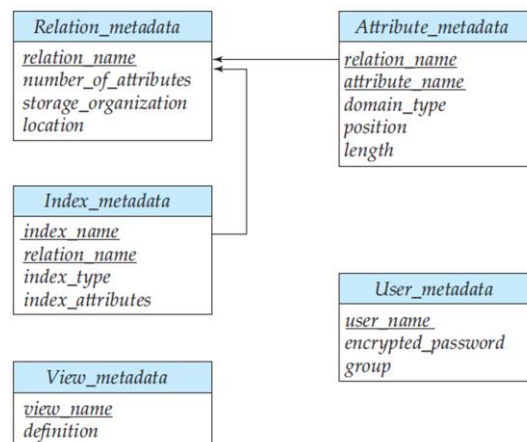


Figure 3.2: Relational schema representing system metadata

Data dictionaries also include data on the secondary keys, indexes and views. The above could also be extended to the secondary key, index as well as view information by defining the secondary key, indexes and views. Data dictionaries do not contain any actual data from the database, it contains only book-keeping information for managing it. Without a data dictionary, however, a database management system cannot access data from the database.

The Database Library is built on a Data Dictionary, which provides a complete description of record layouts and indexes of the database, for validation and efficient data access. The data dictionary can be used for automated database creation, including building tables, indexes, and referential constraints, and granting access rights to individual users and groups. The database dictionary supports the concept of Attached Objects, which allow database records to include compressed BLOBs (Binary Large Objects) containing images, texts, sounds, video, documents, spreadsheets, or programmer-defined data types.

3.3 Functions of Data Dictionary:

The data dictionary stores useful metadata, such as field descriptions, in a format that is independent of the underlying database system.

1. Ensuring efficient data access, especially with regard to the utilization of indexes,
2. Partitioning the database into both logical and physical regions,
3. Specifying validation criteria and referential constraints to be automatically enforced,
4. Supplying pre-defined record types for Rich Client features, such as security and administration facilities, attached objects, and distributed processing (i.e., grid and cluster supercomputing).

3.3.1 Features of system Catalog

A comprehensive data dictionary product will include possess the following features:

5. support for standard entity types (elements, records, files, reports, programs, systems, screens, users, terminals, etc.), and their various characteristics (e.g., for elements, the dictionary might maintain Business name, Business definition, name, Data type, Size, Format, Range(s), Validation criteria, etc.)
6. support for user-designed entity types (this is often called the “extensibility” feature); this facility is often exploited in support of data modelling, to record and cross-reference entities, relationships, data flows, data stores, processes, etc.
7. the ability to distinguish between versions of entities (e.g., test and production)

8. enforcement of in-house standards and conventions.
9. comprehensive reporting facilities, some of the reports include:
 1. detail reports of entities
 2. summary reports of entities
 3. component reports (e.g., record-element structures)
 4. cross-reference reports (e.g., element keyword indexes)
 5. where-used reports (e.g., element-record-program cross-references).
10. a query facility, both for administrators and casual users, which includes the ability to perform generic searches on business definitions, user descriptions, synonyms, etc.
11. language interfaces, to allow, for example, standard record layouts to be
12. automatically incorporated into programs during the compile process.
13. automated input facilities (e.g., to load record descriptions from a copy library).
14. security features
15. adequate performance tuning abilities
16. support for DBMS administration, such as automatic generation of DDL

Limitations of System Catalog

1. A system catalog is a useful management tool, but it also pose several challenges. It needs careful planning. We would need to define the exact requirements designing its contents, testing, implementation and evaluation.
2. The cost of a system catalog includes not only the initial price of its installation and any hardware requirements, but also the cost of collecting the information entering it into the DDS, keeping it up-to date and enforcing standards.
3. The use of a system catalog requires management commitment, which is not easy to achieve, particularly where the benefits are intangible and long term.

System Catalog in ORACLE

Meta data - data dictionary: Information about schema objects: tables, indexes, views, triggers,

Meta data are divided into three levels:

- information for objects owned by a user
- information for objects owned by a user as well as the objects that the user has been granted access to
- information about all database objects

Meta data are divided into three levels - three kinds of views:

- view name prefixed with USER
- view name prefixed with ALL
- view name prefixed with DBA

3.3.1 Views with the Prefix USER

The views most likely to be of interest to typical database users are those with the prefix USER. These views are as follows:

- refer to the user's own private environment in the database, including information about schema objects created by the user, grants made by the user, and so on,
- display only rows pertinent to the user,
- have columns identical to the other views, except that the column OWNER is implied,
- return a subset of the information in the ALL views,
- can have abbreviated PUBLIC synonyms for the sake of convenience.

For example, the following query returns all the objects contained in a schema:

```
SELECT object_name, object_type FROM USER_OBJECTS;
```

3.3.2 Views with the Prefix ALL

Views with the prefix ALL refer to the user's overall perspective of the database. These views return information about schema objects to which the user has access through public or explicit grants of privileges and roles, in addition to schema objects that the user

owns. For example, the following query returns information on all the objects to which the user has access:

```
SELECT owner, object_name, object_type FROM ALL_OBJECTS;
```

Views with the Prefix DBA

Views with the prefix DBA show a global view of the entire database. Synonyms are not created for these views, because DBA views should be queried only by administrators. Therefore, to query DBA views, administrators must prefix the view name with its owner, SYS, as in the following:

```
SELECT owner, object_name, object_type
```

3.3.2.3 Role of System Catalog in Database Administration

Database administration is a specialized database activity that is performed by a database administrator. The system catalogue has an important role to play in the database administration. Some of the key areas where the system catalogue helps the database administrator are defined below:

1. **Enforcement of Database Integrity:** System catalogue is used to store information on keys, constraints, referential integrity, business rules, triggering events etc. on various tables and related objects. Thus, integrity enforcement would necessarily require the use of a data dictionary.
2. **Enforcement of Security:** The data dictionary also stores information on various users of the database systems and their access rights. Thus, enforcement of any security policy has to be processed through the data dictionary.
3. **Support for Database System Performance:** The data dictionary contains information on the indexes, statistics etc. Such information is very useful for query optimization. Also such information can be used by the database administrator to suggest changes in the internal schema.
4. Data dictionary can also support the process of database application development and testing as they contain the basic documentation while the systems are in the process of being developed.

3.4. Conclusion

The catalogue should normally be self-describing i.e. it should include entries describing the catalogue relvars themselves. System catalog plays a significant role in database activities and in database administration by enforcing integrity, enhancing security and supports database system performance.

3.5 Summary:

This unit provides a detailed view of a data dictionary in a DBMS. The data dictionary is one of the most important implementation tools in a database system. The system catalogue provides all the information on various entities, attributes, database statistics etc. It is a very useful tool if implemented actively in a database system. However, active implementation of a data dictionary is costly.

In this unit we have discussed concepts related to data dictionary and its use in oracle by the different types of users. We have also presented information on the data dictionary system and its advantages and disadvantages. We have provided a brief introduction to system catalogue in distributed systems and how data dictionary is useful in system administration.

3.6. Tutor-Marked Assignment (TMA)

1. What is a database catalogue system?
2. Describe the contents of a system Catalog
3. Mention the various benefits of system catalog in the administration of database
4. Discuss in detail, the features of the system catalog
5. List the disadvantages of a data dictionary

3.7 References and Further Readings

- Atzeni, P., Ceri, S., Paraboschi, S., & Torlone, R. (1999). *Database systems: concepts, languages & architectures* (Vol. 1). London: McGraw-Hill.
- Lorents, A. C., & Morgan, J. N. (1997). *Database Systems: Concepts, Management and Applications*. Harcourt Brace College Publishers.
- Cho, H. (1997, August). Catalog management in heterogeneous distributed database systems. In *1997 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing, PACRIM. 10 Years Networking the Pacific Rim, 1987-1997* (Vol. 2, pp. 659-662). IEEE.
- Kim, D., Lee, S. G., Chun, J., Park, S., Oh, J., Shillimdong, K., & San, Y. N. (2003). Catalog management in e-Commerce systems. *Proceeding of Comp. Sci. & Technology*.